

Aula 06

Engenharia de Software

O Manifesto Ágil, o debate ágil × dirigido por plano, e as técnicas de Extreme Programming.

ITEC · ENGENHARIA DA COMPUTAÇÃO · UFPA



FCT FACULDADE DE
ENGENHARIA DA
COMPUTAÇÃO E
TELECOMUNICAÇÕES



Agenda da Aula

01

O Manifesto Ágil

Valores, princípios e a origem em Snowbird, 2001.

02

Debate: Ágil × Dirigido por Plano

Vocês defendem o ágil contra o professor — de propósito, contra vocês.

03

Técnicas de XP

Histórias de usuário, refatoração, pair programming, integração contínua — e uma oficina para o P1.

04

Síntese & Verificação

Exercícios graduados e checagem final de compreensão.

- 1** LEMBRAR
Enunciar os quatro valores e os doze princípios do Manifesto Ágil (2001) e sua origem histórica.
- 2** ENTENDER
Explicar por que «**ágil** × **dirigido por plano**» é um falso dicotômico — reconhecendo elementos de cada lado no espectro.
- 3** ENTENDER
Descrever as quatro técnicas centrais de XP e o princípio comum que as une: feedback rápido.
- 4** APLICAR
Escrever histórias de usuário válidas para o próprio projeto, aplicando o critério **INVEST**.
- 5** ANALISAR
Argumentar a favor e contra a adoção ágil em um cenário dado, evitando adesão acrítica.

BLOCO 1

O Manifesto Ágil

De onde vêm os quatro valores

01

Snowbird, Utah — fevereiro de 2001

Dezessete desenvolvedores experientes — não iniciantes, não acadêmicos puros — se reuniram numa estação de esqui para discutir algo que já praticavam isoladamente: métodos «leves» (lightweight), em oposição aos processos pesados e burocráticos então dominantes.

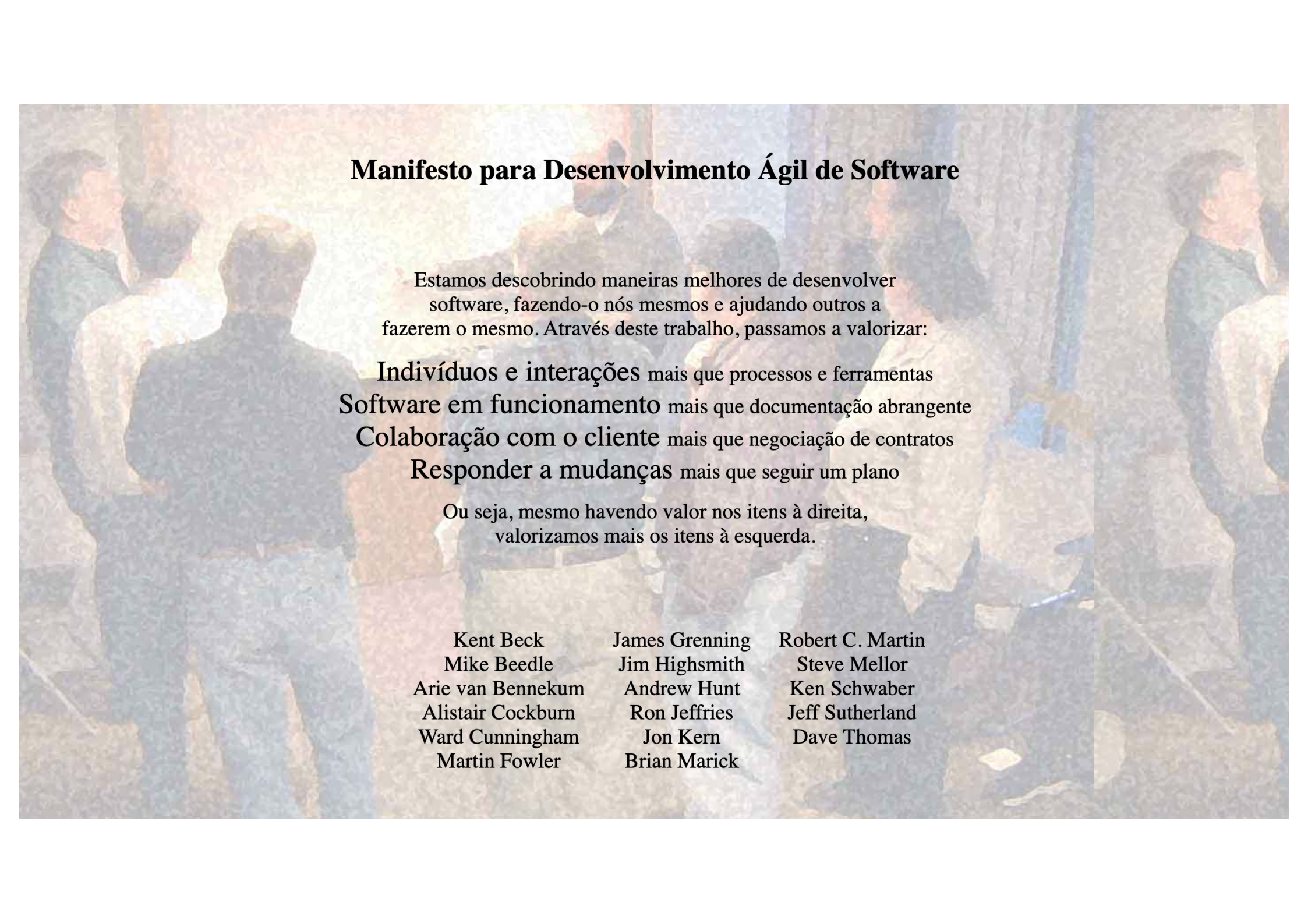
QUEM ESTAVA LÁ

- Kent Beck e Ward Cunningham — Extreme Programming
- Ken Schwaber e Jeff Sutherland — Scrum
- Alistair Cockburn — Crystal
- Martin Fowler, Robert C. Martin, entre outros

O QUE PRODUZIRAM

- 4 valores + 12 princípios, menos de uma página
- Publicado em agilemanifesto.org
- Não inventaram práticas novas — nomearam e unificaram um movimento já disperso

Ponto-chave: XP e Scrum já existiam antes de 2001. O manifesto não é a origem das práticas — é o documento que deu a elas um vocabulário comum.



Manifesto para Desenvolvimento Ágil de Software

Estamos descobrindo maneiras melhores de desenvolver software, fazendo-o nós mesmos e ajudando outros a fazerem o mesmo. Através deste trabalho, passamos a valorizar:

Indivíduos e interações mais que processos e ferramentas
Software em funcionamento mais que documentação abrangente
Colaboração com o cliente mais que negociação de contratos
Responder a mudanças mais que seguir um plano

Ou seja, mesmo havendo valor nos itens à direita,
valorizamos mais os itens à esquerda.

Kent Beck
Mike Beedle
Arie van Bennekum
Alistair Cockburn
Ward Cunningham
Martin Fowler

James Grenning
Jim Highsmith
Andrew Hunt
Ron Jeffries
Jon Kern
Brian Marick

Robert C. Martin
Steve Mellor
Ken Schwaber
Jeff Sutherland
Dave Thomas

«Valorizamos... mais que...»

1. **Indivíduos e interações** mais que processos e ferramentas
2. **Software em funcionamento** mais que documentação abrangente
3. **Colaboração com o cliente** mais que negociação de contratos
4. **Responder a mudanças** mais que seguir um plano

Texto oficial do manifesto: «Ou seja, embora haja valor nos itens à direita, valorizamos mais os itens à esquerda.» A leitura errada mais comum: tratar isso como «os itens à direita não servem para nada».

ANALOGIA

O manifesto não é um mapa — uma rota fixa e completa. É uma bússola: diz a direção a preferir quando as duas opções competem por tempo e atenção, mas não tira a segunda opção do bolso.

Verdadeiro ou falso?

«O Manifesto Ágil afirma que documentação e processos não têm valor algum.»

Falso.

O manifesto prioriza um lado sobre o outro — não elimina o lado de menor prioridade. Documentação e processos continuam tendo valor; só não são o item que vence quando os dois competem por tempo e atenção da equipe.

Os doze princípios, em quatro grupos

ENTREGA & CLIENTE

- Satisfazer o cliente via entrega contínua e adiantada
- Aceitar mudanças mesmo tarde no desenvolvimento
- Entregar com frequência (semanas, não meses)
- Cliente e time trabalham juntos diariamente

PESSOAS & EQUIPE

- Indivíduos motivados, com confiança e suporte
- Conversa face a face é o método mais eficiente
- Equipes auto-organizadas geram o melhor design

EXCELÊNCIA TÉCNICA

- Software funcionando é a medida primária de progresso
- Ritmo sustentável, indefinidamente
- Atenção contínua à excelência técnica

SIMPLICIDADE & REFLEXÃO

- Simplicidade: maximizar o trabalho não realizado
- Em intervalos regulares, refletir e ajustar

BLOCO 3 · ATIVIDADE PRÁTICA

Técnicas de XP

Extreme Programming (Kent Beck) leva os valores do manifesto para dentro do código — não só da gestão.

02

Um princípio, quatro técnicas

XP é o método ágil mais radical em técnicas de engenharia — não só em gestão de projeto. As quatro técnicas centrais (histórias de usuário, refatoração, programação em par, integração contínua) compartilham um único princípio: reduzir o tempo entre um erro — de código ou de requisito — acontecer e ser percebido. É o mesmo mecanismo da entrega incremental da Aula 05, agora na escala do código do dia a dia.

ANALOGIA

As quatro técnicas são como uma equipe de F1 num pit stop de 2 segundos: checagem constante da aerodinâmica (refatoração), dois engenheiros de rádio monitorando em tempo real (pair programming), troca de pneu imediata a cada problema (integração contínua) e um plano de corrida em cartões que muda a cada volta, não fixado antes da largada (histórias de usuário). Tudo orientado a reagir rápido sem quebrar o carro.

Template e critério INVEST

Como [papel], quero [funcionalidade], para que [benefício].

Independente — não depende de outra história para fazer sentido

Negociável — é um convite à conversa, não um contrato fechado

Valiosa — entrega benefício perceptível ao usuário

Estimável — a equipe consegue dimensionar o esforço

Small (pequena) — cabe em um incremento curto

Testável — existe um critério objetivo de «pronto»

Diferença de caso de uso: a história é deliberadamente leve — um convite à conversa com o cliente, não um documento que tenta prever tudo antecipadamente.

Uma história ruim, uma história boa

RUIM

«O sistema deve gerenciar o laboratório.»

Viola INVEST: não tem papel específico, não é pequena, não é testável — «gerenciar» não tem critério de pronto.

BOA

«Como técnico do LABCOM1, quero registrar o empréstimo de um osciloscópio com nome do aluno e data de devolução, para que eu saiba quem está com o equipamento e cobrar atrasos.»

Papel específico, pequena, testável (nome + data + devolução), valor claro (cobrar atraso).

Este é o mesmo cenário do exemplo trabalhado da Aula 03 (controle de empréstimo de osciloscópios) — agora reescrito como história de usuário.

Refatoração e Programação em Par

REFATORAÇÃO

Refatoração

Melhorar a estrutura interna do código **sem mudar o comportamento externo observável**. Não é reescrever do zero — são passos pequenos e reversíveis (extrair método, renomear, remover duplicação).

Ligação com dívida técnica: refatorar é pagar a dívida antes que os juros (retrabalho) fiquem impagáveis.

PAIR PROG.

Pair Programming

Dois desenvolvedores, uma estação: **driver** digita, **navigator** revisa e pensa estrategicamente — papéis trocados com frequência.

Benefício: revisão contínua, disseminação de conhecimento. **Custo real:** dois salários no mesmo problema — trade-off a justificar, não a aceitar por dogma.

Integração Contínua

Integrar e testar o código de toda a equipe várias vezes ao dia, com um build automatizado que falha alto e cedo — em vez de acumular conflitos e defeitos até uma integração final tensa.

ANALOGIA

Lembram da Fábrica de Aviões de Papel? Quanto mais tarde um defeito é descoberto, mais caro ele é. Integração contínua é a validação acontecendo a cada poucas horas, não a cada poucos meses — o mesmo princípio da curva de custo da mudança, aplicado ao código todo dia.

Escrevam 3 histórias do seu projeto

Em equipe, escrevam **3 histórias de usuário reais** do domínio do projeto de vocês, aplicando o template e conferindo cada uma contra o checklist INVEST.

Por que agora: essas histórias entram direto no backlog do P1 (entrega 23/09 — a próxima aula!). Vocês precisam de pelo menos 20 histórias no P1; hoje vocês saem com pelo menos 3 já revisadas.

UMA história de cada equipe será lida em voz alta; a turma aponta 1 critério INVEST que ainda está fraco.